Lab 5 – Shared Memory

Problem 2:

Write a C program that creates a child process and uses shared memory as a mean of communication between the processes. Your program should:

1. Allocate a shared array of 3 strings with a maximum string size of 50
2. Fork a child that reads 3 strings from the user and save them in the array
3. The parent process should wait for the child process to terminate and then transforms all the characters from lower to upper case. The parent should then display every string and its length.

Note:

- *In order to have the three strings in one array, you may need first to allocate a shared memory for the whole array char **shared_array and then allocate shared memory for each element shared array[i] in the array. Also, free the shared memory at the end of your program.*

Sample Output:

From child:
Please enter String 1: Hello
Please enter String 2: Computer
Please enter String 3: Science

From Parent:

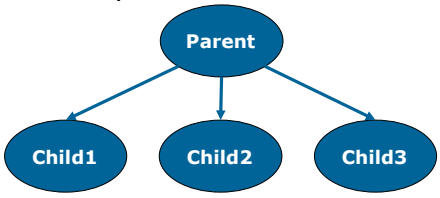
*** String 1 ***
HELLO
Length = 5

*** String 2 ***
COMPUTER
Length = 8

*** String 3 ***
SCIENCE
Length = 7

Problem 3:

Write a C program that will create three processes that share an array of *N* elements as follows:



- Scan the length of the array of integers *N* from the user and create a shared array between the different processes
- Child 1 will scan the integers from the user and fill the array
- Child 2 will multiply the array elements by 10
- Child 3 will deduct 2 from every element
- The parent will then compute the sum of the elements of the array. The parent should then expand the existing memory mapping for the array to add the sum at the last position of the array. *Note:*
- *You may need to use the mremap() function (void * mremap(void * old address, size t old size, size t new _size, int flags);) which expands (or shrinks) an existing memory mapping. You may set the flag to 0. In order to use the mremap() method, you need to add "#define GNU _SOURCE" before including any headers.*
- *You need to implement a function printArray to display the elements of the array. Also, free the shared memory at the end of your program.*

Sample Output:

```
Please enter the size of the array:
3
*** From Child 1: ***
Please enter 3 integers:
1
2
3
1 2 3
*** From Child 2: ***
10 20 30
*** From Child 3: ***
8 18 28
*** From Parent: ***
8 18 28 54
```

Problem 4:

Write a C program that creates 3 processes (P1, P2, and P3)(Parent and 2 children). The 3 processes should increment the value of an integer declared as a shared memory using mmap() as follows:

- P1 (Child 1) should loop to increment the integer from 0 to 10000
- P2 (Child 2) should loop to increment the integer from 10000 to15000
- P3 (Parent) should loop to increment the integer from 15000 to 20000
- Each process should print the value of the integer before exiting

Note: You are not required to do any sort of synchronization in this question. Consider the order of execution of the different processes by using wait or waitpid methods.

Sample Output:

Child 1 10000
Child 2 15000
Parent 20000